1. Objective

Generate a complete, buildable embedded application demonstrating **closed-loop field-oriented control of a three-phase PMSM**, using FreeRTOS on the **P-NUCLEO-IHM03 kit**.

The application shall start the motor, transition into sensorless closed-loop operation, regulate speed, support controlled stopping and reversal, report telemetry, and disable motor outputs on faults.

Closed-loop operation must use:

- Measured phase currents.
- Estimated rotor electrical angle and speed.
- Independent d-axis and q-axis current controllers.
- An outer speed controller.

A rotating angle calculated only from the requested frequency does **not** satisfy the closed-loop requirement.

2. Hardware target

Item	Target
Control board	NUCLEO-G431RB
MCU	STM32G431RB, Cortex-M4F, 170 MHz
Memory budget	128 KB flash, 32 KB SRAM
Power board	X-NUCLEO-IHM16M1
Driver	STSPIN830
Motor	GBM2804H-100T PMSM, seven pole pairs
Nominal supply	Kit’s 12 V DC, 2 A supply
Current sensing	Three-shunt configuration
User interfaces	B1 button, LD2 LED, speed potentiometer
Programming/debugging	STLINK-V3E
Console	ST-LINK Virtual COM Port, 115200 baud, 8N1

Use the board’s supported three-PWM driver mode. Verify MODE configuration, fitted resistor links, current-sense routing, and fault routing against the applicable board revision.

Do not assume all hardware-supported features are enabled by default.

Candidate pin mapping

The existing project uses the following mapping. Verify it before adopting it:

Signal	Pin/peripheral
Phase U PWM	PA8 / TIM1_CH1
Phase V PWM	PA9 / TIM1_CH2
Phase W PWM	PA10 / TIM1_CH3
Phase enables	PB13 / PB14 / PB15
Driver EN/FAULT	PB12 and/or PA11, depending on fitted routing
Phase U current	PA1 / ADC1_IN2
Phase V current	PB1 / ADC1_IN12
Phase W current	PB0 / ADC1_IN15
Bus voltage	PA0 / ADC1_IN1
Potentiometer	PC2 / ADC1_IN8
Board temperature input	PC4 / ADC1_IN5
Button	PC13
LED	PA5
UART	PA2 / PA3 / USART2

If this mapping cannot support the required synchronized current sampling, explain the peripheral constraints and implement a documented valid acquisition scheme.

3. Implementation constraints

- Use **FreeRTOS** with static allocation for application tasks, queues, and synchronization objects.
- Use STM32CubeG4 HAL/LL or CMSIS register access consistently.
- Keep the control algorithm modular and independently testable.
- Use the MCU's FPU where appropriate.
- No dynamic allocation after initialization.
- No blocking UART, formatting, logging, or mutex acquisition in the fast control interrupt.

- Select and document compatible interrupt priorities and FreeRTOS interrupt API rules.
- Document ownership of SysTick and the HAL time base, if HAL is used.
- Pin dependency versions and provide reproducible build instructions.
- State whether ST Motor Control SDK components are used. Include required dependencies and licensing information.
- Do not provide placeholder implementations for essential control, acquisition, or protection functions.

4. Operating modes and state machine

Implement these explicit states:

State	Required behavior
BOOT	Configure hardware with motor outputs disabled
SELF_TEST	Validate configuration, supply, ADC operation, and fault input
CALIBRATION	Determine current-sensor offsets with the bridge disabled
IDLE	Accept commands; motor outputs disabled
ALIGN	Apply a limited stationary current vector
STARTUP	Run current-regulated forced-angle acceleration
HANDOVER	Validate observer lock and blend to estimated angle
RUN	Regulate speed using sensorless closed-loop FOC
STOPPING	Decelerate within current and bus-voltage limits
FAULT	Disable outputs and retain diagnostic information

Requirements:

1. Power-on shall never automatically start the motor.
2. A start request shall be accepted only after calibration and self-test succeed.
3. Every startup stage shall have a timeout.
4. Startup failure shall enter FAULT.
5. Fault clearing shall return to IDLE without restarting.
6. All state transitions shall be owned by one supervisor.
7. A fault shall take precedence over a normal transition.
8. Invalid commands shall return a reason.
9. Reversal shall decelerate to a validated low-speed condition before starting in the opposite direction.
10. A software emergency-disable command shall immediately disable the bridge; distinguish it from controlled stopping.

5. Current acquisition

Implement PWM-synchronized current acquisition suitable for closed-loop FOC.

Requirements:

- Use timer-triggered ADC conversion rather than a software-triggered scan from an RTOS task.
- Sample within valid low-side shunt conduction windows.
- Account for switching blanking, amplifier settling, ADC sampling time, and conversion latency.
- Explain whether phase currents are sampled simultaneously or sequentially.
- Use at least two valid phase-current measurements per control update; reconstruct the third using:
  `iu + iv + iw = 0`.
- If all three are valid, use their sum as a plausibility diagnostic.
- Mark samples with validity and freshness information.
- Handle small conduction windows by limiting modulation, adapting sampling, or another documented method.
- Do not silently use invalid samples as actual current feedback.
- Define the response to isolated invalid samples and repeated acquisition failures.
- Acquire bus voltage, potentiometer, and temperature separately without disrupting current sampling.

At boot, average a configurable number of samples to calibrate each current offset. Reject offsets outside configurable plausible ranges.

Keep ADC scaling, polarity, gain, and offset parameters in one board configuration module.

6. Fast FOC current loop

Target **20 kHz center-aligned PWM**, with one current-control update per PWM period.

Each control update shall:

1. Validate the acquired samples.
2. Convert ADC measurements to phase currents in amperes.
3. Perform the Clarke transform.
4. Obtain the selected electrical angle.
5. Perform the Park transform.
6. Run d-axis and q-axis current PI controllers.
7. Apply voltage-vector saturation and anti-windup.
8. Perform the inverse Park transform.
9. Generate SVPWM duty cycles.
10. Apply pulse-width and sampling-window constraints.
11. Update PWM compare registers coherently.
12. Update the sensorless observer using time-aligned current and applied-voltage information.

Additional requirements:

- Use `Id_reference = 0` during normal operation initially.
- The speed controller shall generate the bounded `Iq_reference`.
- Use measured bus voltage when calculating available voltage and duty cycles.
- Account for output saturation when estimating applied voltage.
- Reset or initialize controller integrators appropriately during state transitions.
- Define transform conventions, phase ordering, angle direction, and current sign.
- Include compensation or explain the limitations of driver dead time and inverter voltage error.
- Field weakening is outside the initial scope.

7. Sensorless estimation and startup

Implement a documented sensorless estimator, such as a back-EMF observer with PLL.

The estimator shall output:

- Electrical angle.
- Electrical speed.
- Mechanical speed.
- Lock/confidence status.
- Diagnostic residuals or equivalent quality metrics.

Use:

`mechanical_rpm = electrical_rad_per_second × 60 / (2π × 7)`.

Required startup sequence:

1. Apply bounded d-axis alignment current.
2. Hold alignment for a configurable duration.
3. Accelerate using a forced electrical angle while regulating current.
4. Run the observer during acceleration.
5. Evaluate lock using configurable criteria sustained over a defined interval.
6. Blend forced and estimated angles without an abrupt jump.
7. Enter RUN only after successful handover.

Do not assume a back-EMF observer works reliably at standstill or at every low speed.

Specify a validated closed-loop speed range. Requests below the supported sensorless floor shall receive an explicit response or invoke a documented fallback mode. Telemetry must distinguish forced-angle operation from closed-loop operation.

Do not invent motor resistance, inductance, flux linkage, or observer gains. Provide a commissioning procedure and mark unverified values clearly. Prevent normal motor startup when essential parameters are missing or invalid.

8. Speed regulation

Run the speed controller at a nominal **1 kHz**.

Requirements:

- Compare requested speed with estimated actual speed.
- Generate a bounded q-axis current request using PI control.
- Include anti-windup and configurable speed filtering.
- Apply acceleration and deceleration limits.
- Support forward and reverse rotation.
- Keep requested speed, ramped reference, and estimated speed separate.
- Define behavior during saturation, observer unlock, and stopping.

Provide configurable speed, current, and acceleration limits.

Do not use the motor's maximum listed speed or current as the default operating target. Provide a conservative commissioning profile with justified limits.

Potentiometer mode

- Filter and validate the input.
- Map it into the validated speed range.
- Define dead zone and zero-speed behavior.
- Select console or potentiometer ownership explicitly.
- Prevent conflicting setpoint sources.

9. RTOS architecture

Keep the fast current loop in a deterministic hardware interrupt. Do not schedule the 20 kHz FOC loop using RTOS delays.

Implement these tasks:

Task	Nominal rate	Responsibility
Motor supervisor and speed control	1 kHz	State transitions, speed PI, reference generation
Command processing	Event driven	UART parsing and command validation
User interface	50 Hz	Button debounce, potentiometer handling, LED
Telemetry	10 Hz	Publish coherent diagnostic snapshots

Health monitoring	100 Hz	Task health, timing diagnostics, watchdog policy

Requirements:

- Document task priorities, stack sizes, and execution budgets.
- Pass commands through bounded queues.
- Use coherent snapshots or double buffering for task/ISR data exchange.
- Do not rely on `volatile` alone for multi-field consistency.
- Define queue-overflow and telemetry-backpressure behavior.
- Telemetry congestion shall not affect motor control.
- Enable stack-overflow detection and FreeRTOS assertions.
- Feed the watchdog only when required control and task health checks pass.

10. Protection and fault handling

Implement:

- Driver fault detection.
- Software overcurrent detection.
- Bus undervoltage and overvoltage detection.
- ADC timeout and stale-data detection.
- Repeated invalid current-sample detection.
- Startup timeout.
- Observer unlock/loss of synchronism.
- Stall or sustained speed-error detection.
- Control execution overrun detection.
- Watchdog recovery.
- Invalid configuration detection.
- Overtemperature protection if the temperature channel is calibrated; otherwise report that this protection is unavailable.

For each fault, document:

- Threshold and units.
- Debounce or persistence time.
- Detection context.
- Output-disable path.
- Latching policy.
- Clear conditions.

Requirements:

- Preserve hardware protection behavior.
- Critical driver/current faults shall disable outputs without waiting for an RTOS task.
- Use a hardware break path if the verified board routing supports it; otherwise document the interrupt-based path and its measured latency.

- Treat the driver's shared EN/FAULT net according to its electrical behavior.
- Prevent pull-ups, GPIO modes, or firmware output writes from masking driver faults.
- Fault records shall capture state, timestamp, bus voltage, currents, speed, and observer status.
- Fault clearing shall require the underlying condition to be resolved.
- Define overvoltage handling during regenerative deceleration. Do not assume the supplied adapter can absorb regenerated energy.

11. User interface

Use a line-oriented serial protocol with explicit commands:

```
None
help
status
start
stop
disable
fault clear
speed 300
direction forward
direction reverse
source console
source pot
```

Requirements:

- Commands shall return success or an explanatory error.
- `start` and `stop` shall be idempotent, not toggles.
- Bound input length and handle malformed or overflowing input.
- `disable` shall immediately disable outputs.
- B1 shall start from IDLE, request controlled stopping while running, and clear a recoverable fault without restarting.
- Implement actual button debounce.

LED behavior:

State	Indication
IDLE	Slow blink
ALIGN / STARTUP / HANDOVER	Faster blink
RUN	Solid

STOPPING	Distinct documented pattern
FAULT	Rapid blink

12. Telemetry

Publish at least:

```
None
timestamp
state
control_mode
fault_code
requested_rpm
ramped_reference_rpm
estimated_rpm
electrical_angle
observer_locked
bus_voltage
iu, iv, iw
id, iq
id_reference, iq_reference
vd, vq
duty_u, duty_v, duty_w
current_sample_valid
control_execution_time
control_overrun_count
```

Provide a documented CSV or machine-readable format.

Clearly label estimated speed. Never report requested or commanded speed as measured speed.

13. Timing and resource requirements

- PWM/current-loop period: **50 µs**.
- Proposed maximum fast-loop execution budget: **35 µs**, including acquisition-related ISR work.
- Complete duty updates before the required timer preload deadline.
- Instrument execution time with DWT or a diagnostic GPIO.
- Report worst-case measurements under telemetry and command load.

- Target flash usage below **115 KB** and SRAM usage below **28 KB**, including stacks.
- Report stack high-water marks.
- Explain deadline and memory-budget failures rather than hiding them.

14. Required deliverables

Generate:

1. Complete source code and build configuration.
2. FreeRTOS configuration and dependency versions.
3. Board/peripheral configuration.
4. FOC transforms, PI controllers, SVPWM, and sensorless estimator.
5. State machine and protection module.
6. UART command parser and telemetry.
7. Central motor/board parameter configuration.
8. Wiring and resistor/jumper verification instructions.
9. Motor-parameter measurement and tuning procedure.
10. Build, flash, and serial-monitor instructions.
11. Host-side algorithm tests.
12. Hardware validation procedure.
13. Known limitations and unresolved assumptions.
14. Build size and timing reports where measurements are available.

Separate algorithm code from hardware access so mathematical functions can be tested on a host.

If hardware is unavailable, explicitly distinguish generated code, compiled code, simulated behavior, and physically validated behavior.

15. Acceptance tests

Build and algorithm tests

- Build successfully for STM32G431RB.
- Confirm linker memory usage.
- Verify Clarke/Park and inverse-transform consistency.
- Verify PI saturation and anti-windup.
- Verify SVPWM output bounds and phase ordering.
- Verify angle wrapping in both directions.
- Verify seven-pole-pair speed conversion.
- Verify current reconstruction and sample validity handling.
- Verify fault precedence over normal state transitions.
- Verify malformed commands and bounded queues.
- Exercise observer behavior using documented simulated signals; identify simulation limitations.

Hardware tests

1. Power-on leaves outputs disabled.
2. Invalid calibration prevents starting.
3. Start performs alignment, acceleration, and validated observer handover.
4. RUN uses estimated angle and current feedback.
5. At several points in the validated speed range, target steady-state speed error is within **±5%**, checked against an independent speed measurement.
6. A documented modest load disturbance recovers toward the reference without exceeding current limits.
7. Speed changes respect configured ramps.
8. Reversal follows the stopping/restart sequence.
9. Fault injection disables outputs through the documented path.
10. Fault clearing does not restart the motor.
11. UART saturation does not cause control deadline failures.
12. Bus-voltage variation within the approved operating range is handled.
13. Repeated starts and stops do not accumulate controller state errors.
14. A proposed **30-minute continuous-run test** completes without unexpected faults, overruns, or unacceptable temperature rise.

Define test load, supply conditions, instrumentation, and pass criteria before claiming acceptance.

16. Required implementation explanation

Before presenting the generated application, explain:

- Verified board mode and routing.
- Current-sampling timing and valid windows.
- Selected estimator and supported speed range.
- Startup and handover criteria.
- Motor parameters that require measurement.
- RTOS/interrupt ownership and priorities.
- Protection paths and output-disable latency.
- Which requirements are implemented and which remain unvalidated.

Do not describe forced-angle operation as sensorless closed-loop FOC. Do not claim hardware validation without measured evidence.